

Code Agent 기본 & Engineering 발전과정 이해

CONTENTS

목차

CHAPTER 1

챕터의 목적

CHAPTER 2

AGENTS.md

CHAPTER 3

Subagent

CHAPTER 4

Skill

CHAPTER 5

MCP

Chapter 01

챕터의 목적

하루만에 익히는 Code Agent

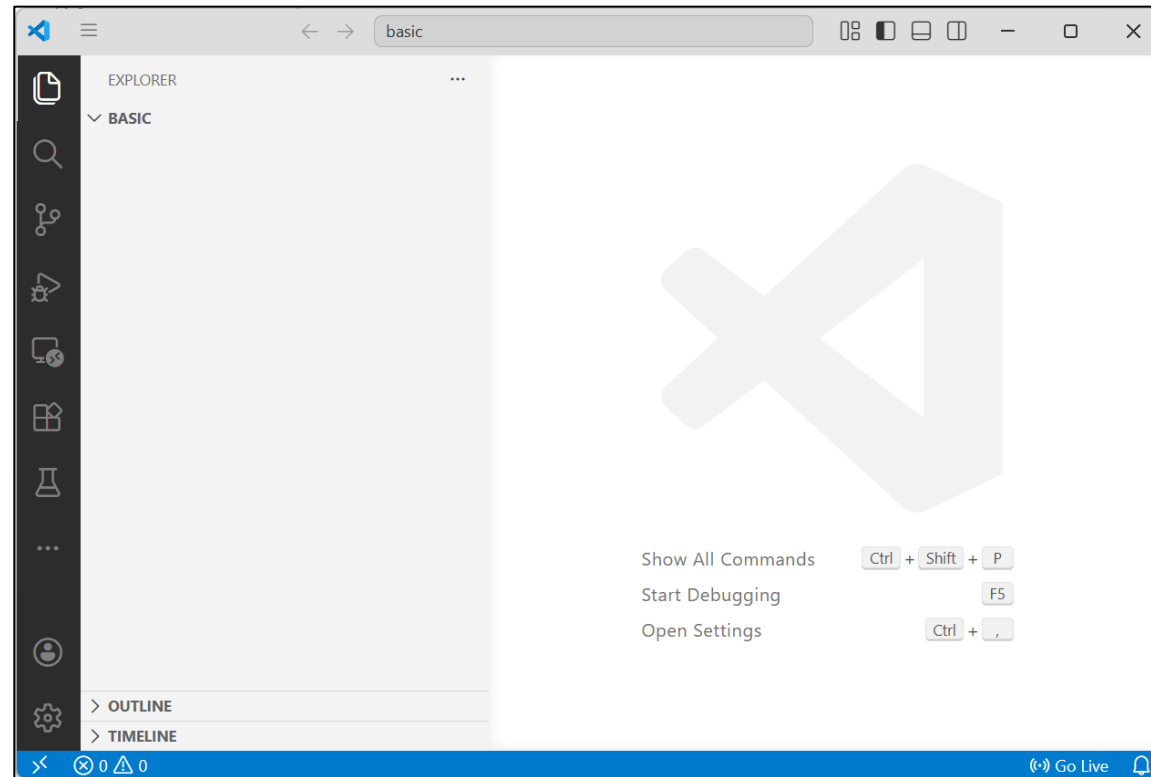
Codex의 기본 사용법을 익힌다.

이 챕터에서는 프로그래밍 소스코드를 다루지 않고, 기본 사용법만 익혀본다.
Agentic Engineering 학습을 위한 최소한의 Code Agent 사용법을 리뷰한다.

커리큘럼

1. AGENTS.md 사용법을 익히고,
2. 간단한 문서를 만들어본다.
3. 문서를 수정해보고, 작업을 취소하는 방법을 배운다.
4. Subagent를 사용해본다.
5. Skill과 MCP 개념을 익히고, 가볍게 다루어본다.

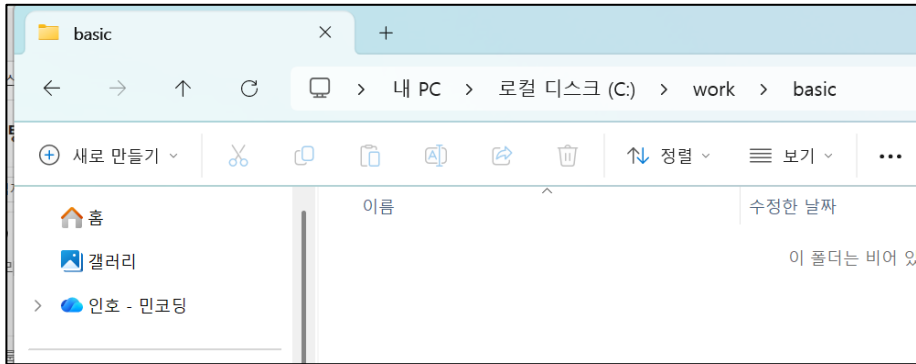
vscode (Visual Studio Code)를 “**문서에디터**” 용도로만 사용
프로그래밍 용도가 아닌, 문서 Editor 용도로만 사용할 예정



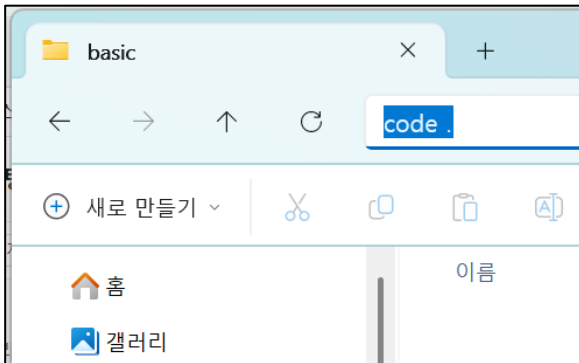
vscode 세팅

폴더 생성 후, **code .** 입력하여 vscode 실행
c:\work\basic 폴더 생성 후, vscode 열기

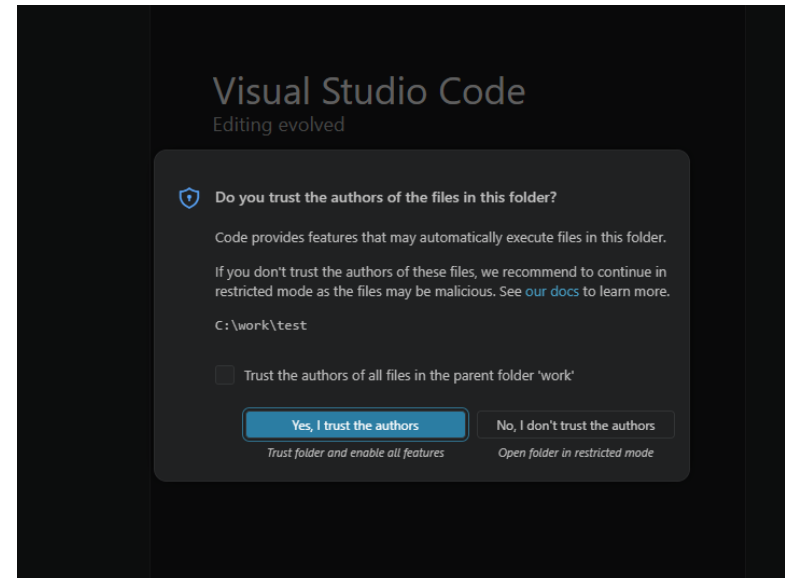
폴더 생성 후 진입



code . 입력



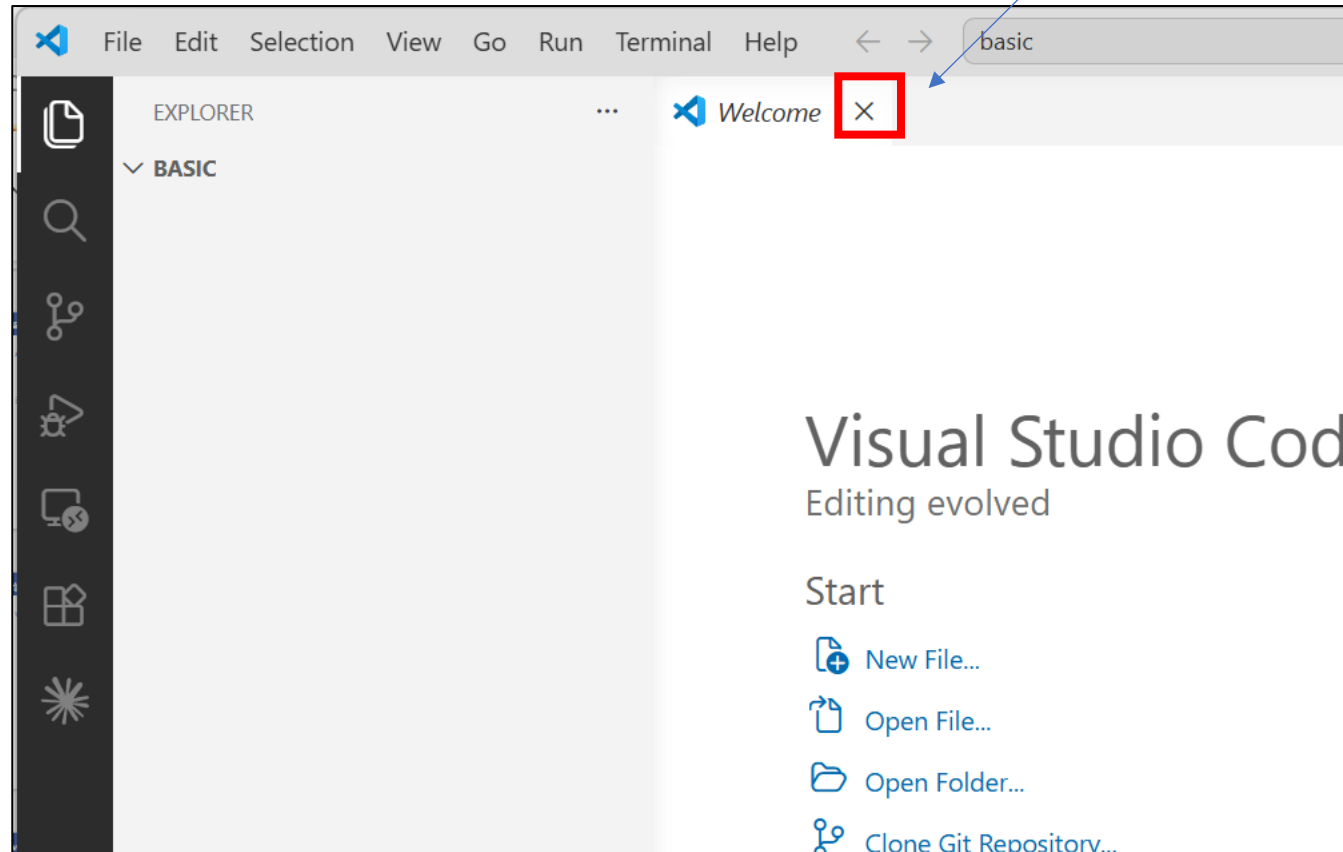
vscode가 실행되면서
이 폴더가 Trust한지 물어봄
이 창이 뜨면 항상 Yes 버튼 누르기



현재 까지 진행상황

BASIC 폴더가 작업 폴더로 선택되어
vscode가 실행되었음

환영합니다
창은 닫아주자



Chapter 02

AGENTS.md

하루만에 익히는 Code Agent

- AGENTS.md 파일이란?

Code Agent에서 사용되는 규칙 / 지침 파일

이곳에 내용을 기입하면, Codex는 사용자의 규칙(지침)으로 간주하고 행동한다.

- 사용처

Code Agent가 항상 참고해야 할 내용을 AGENTS.md에 적는다.

- 매번 프롬프트로 반복 설명할 필요가 없다.
- 이곳에 프로젝트에 대한 설명, 개발 방식, 팀의 규칙 등을 정의한다.

생성해야 하는 위치

현재 열려있는 프로젝트의 Root 폴더에 둔다.

- 루트 폴더에 넣어두면, 프로젝트 전체에 규칙을 적용한다.
- 비유 : 프로젝트의 헌법

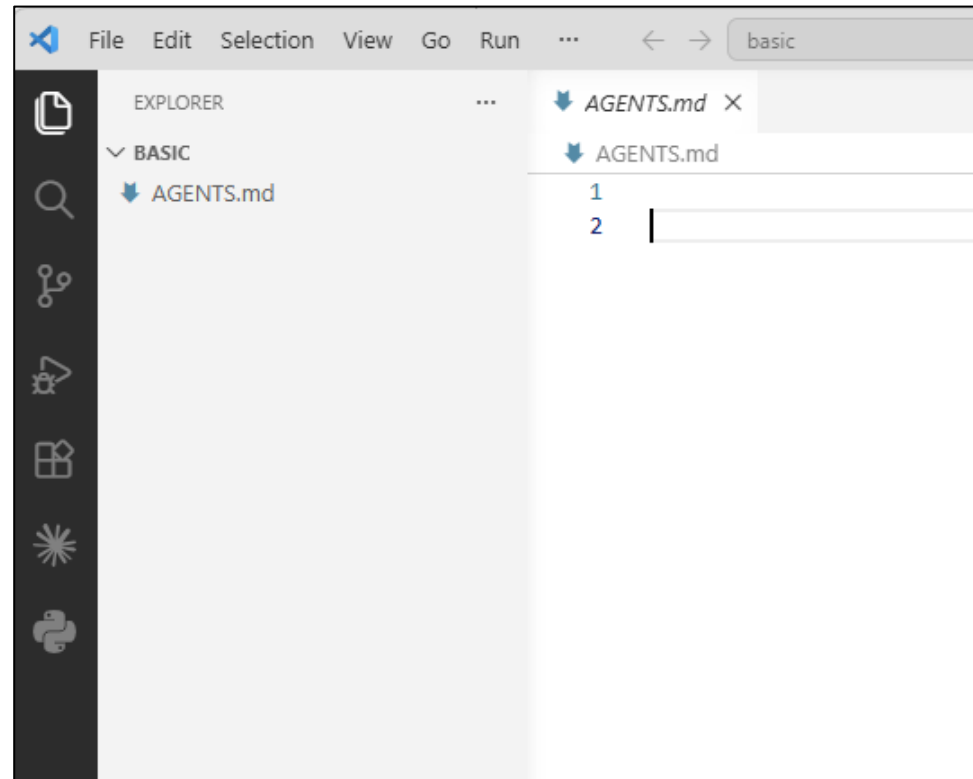
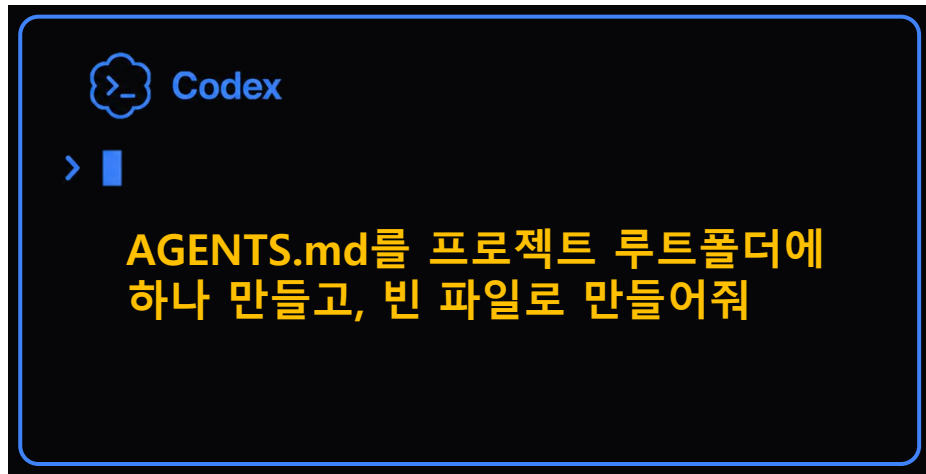
```
/my-project
├── AGENTS.md
├── src/
├── test/
└── README.md
```

경로 구성 예시

AGENTS.md 파일 생성

1. Code Agent에게 지시한다.

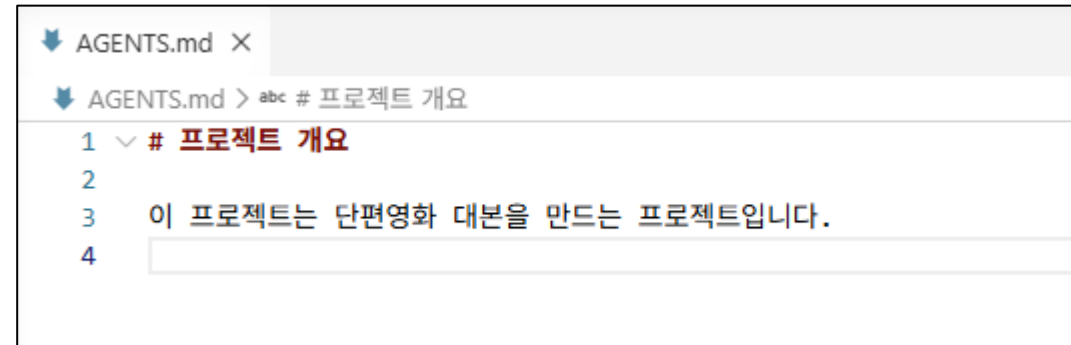
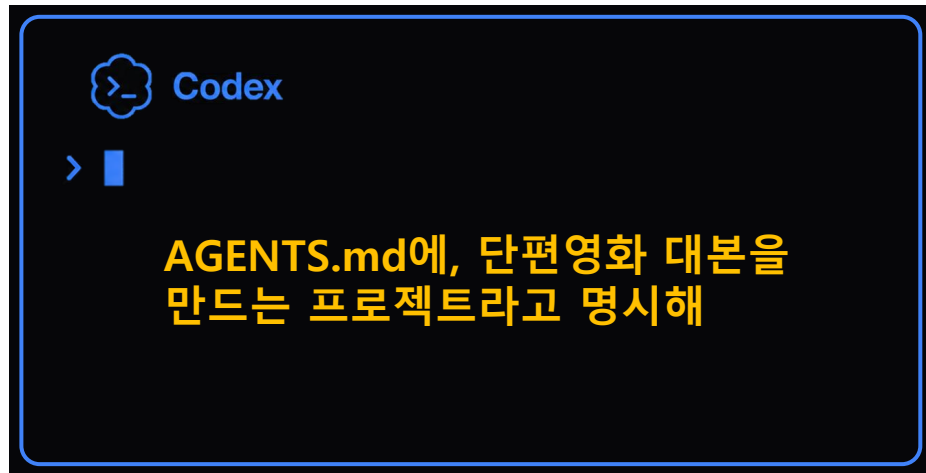
- AGENTS.md 파일 생성



AGENTS.md 기본 내용 작성

2. Code Agent에게 지시한다.

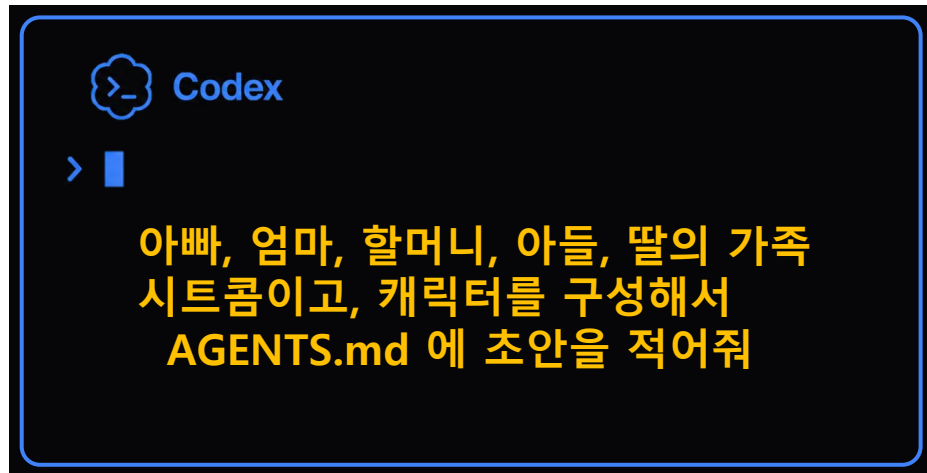
- AGENTS.md에는 프로젝트의 목적을 적어준다.



AGENTS.md 초안 작성 요청

3. Code Agent에게 지시한다.

- AGENTS.md에는 프로젝트 구성요소에 대한 설명을 적는다.
- 프로젝트가 지속적으로 발전하더라도, 변경이 없는 내용을 적는 것이 좋다.



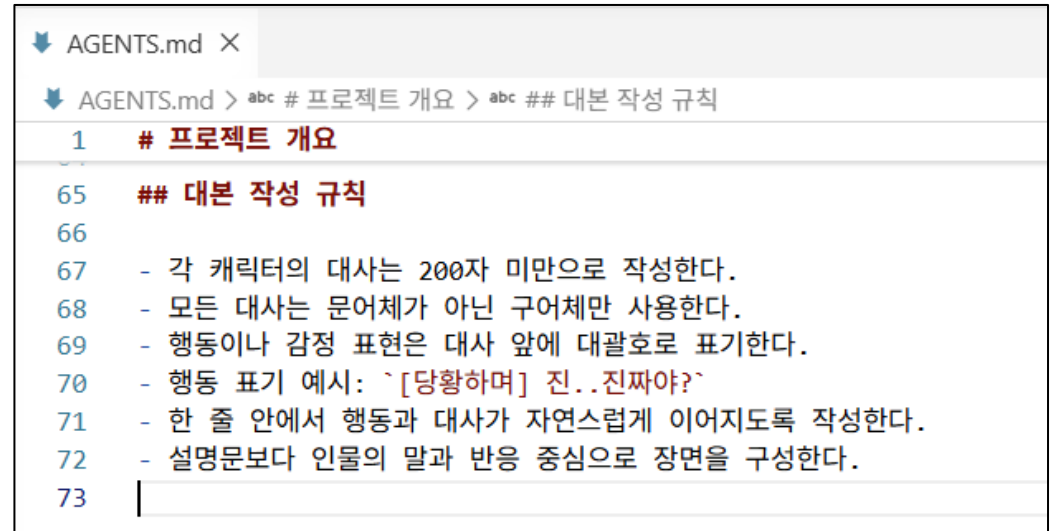
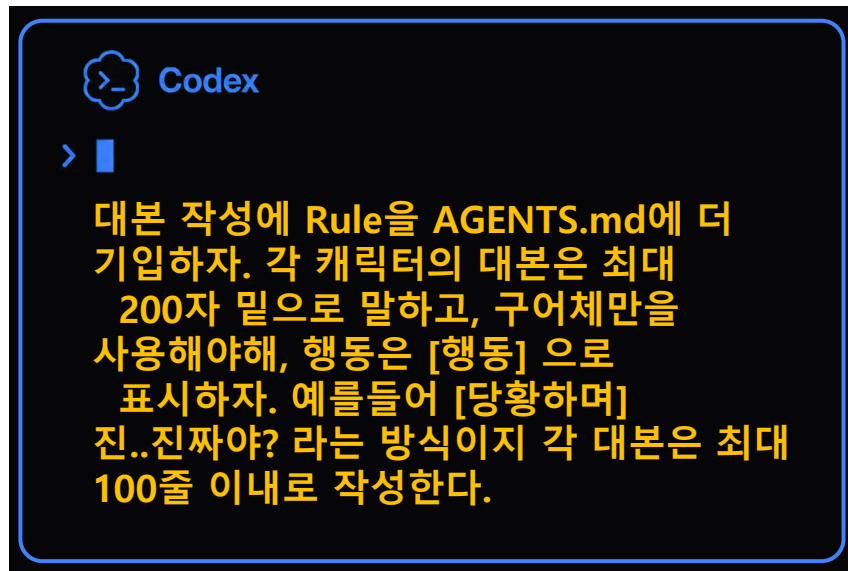
```
AGENTS.md X
AGENTS.md > abc # 프로젝트 개요 > abc ## 시트콤 스타일 가이드 초안
1  ## 프로젝트 개요
2
3  이 프로젝트는 단편영화 대본을 만드는 프로젝트입니다.
4
5  가족 구성은 아빠, 엄마, 할머니, 아들, 딸로 이루어진 가족 시트콤이다.
6
7  ## 프로젝트 초안
8
9  - 장르: 가족 시트콤
10 - 분위기: 따뜻하고 유쾌하지만, 세대 차이와 생활 밀착형 갈등에서 웃음을 만듭니다.
11 - 배경: 한 집에서 함께 살아가는 3세대 가족의 일상
12 - 핵심 주제: 사랑, 오해, 화해, 가족만의 엉뚱한 팀워크
13
14 ## 캐릭터 초안
15
16 ### 아빠
17
18 - 가족의 중심이라고 생각하지만 실제로는 늘 계획이 어긋나는 인물
19 - 허세와 책임감이 동시에 있다.
20 - 사소한 일도 크게 벌이지만, 결국 가족을 가장 먼저 챙긴다.
21 - 갈등 포인트: 체면을 중시하지만 집에서는 자주 권위를 잃는다.
22
23 ### 엄마
24
25 - 집안의 실질적인 균형을 잡는 인물
26 - 현실적이고 눈치가 빠르며, 가족들의 성격을 가장 잘 파악하고 있다.
27 - 단호한 말투 속에 애정이 있다.
28 - 갈등 포인트: 모두를 챙기느라 정작 자기 감정은 뒤로 미룬다.
29
30 ### 할머니
31
32 - 가족 내 최고 어른이자 예상 밖의 승부사
33 - 옛날식 가치관을 말하면서도 필요할 때는 누구보다 유연하다.
34 - 무심한 듯 툭 던지는 말로 분위기를 뒤흔든다.
35 - 갈등 포인트: 간섭이 심한 것 같지만, 사실은 가족 걱정이 많다.
36
37 ### 아들
38
39 - 사고도 치고 말도 많지만 미워할 수 없는 인물
40 - 자신만의 논리로 행동하며 자주 문제를 키운다.
41 - 새로운 유행과 기술에 밝다.
```

Agent의 랜덤성으로 인해, 실제 실습 결과는 이미지와 다를 수 있습니다.
위 실습 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

AGENTS.md 프로젝트 규칙 설정

4. Code Agent에게 지시한다.

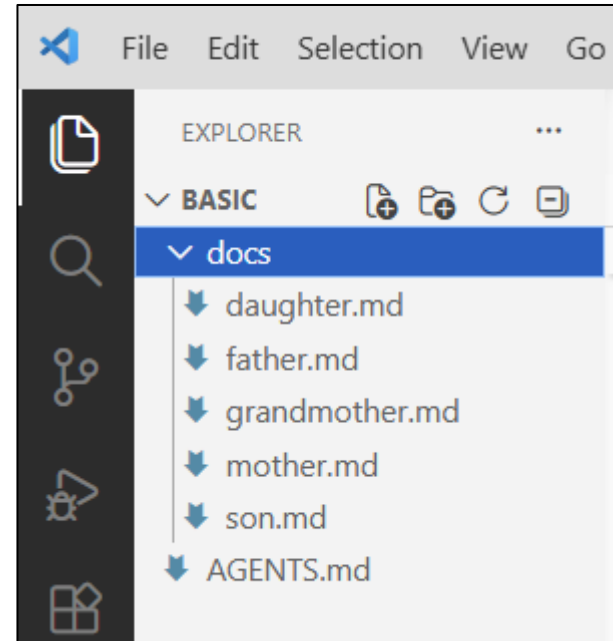
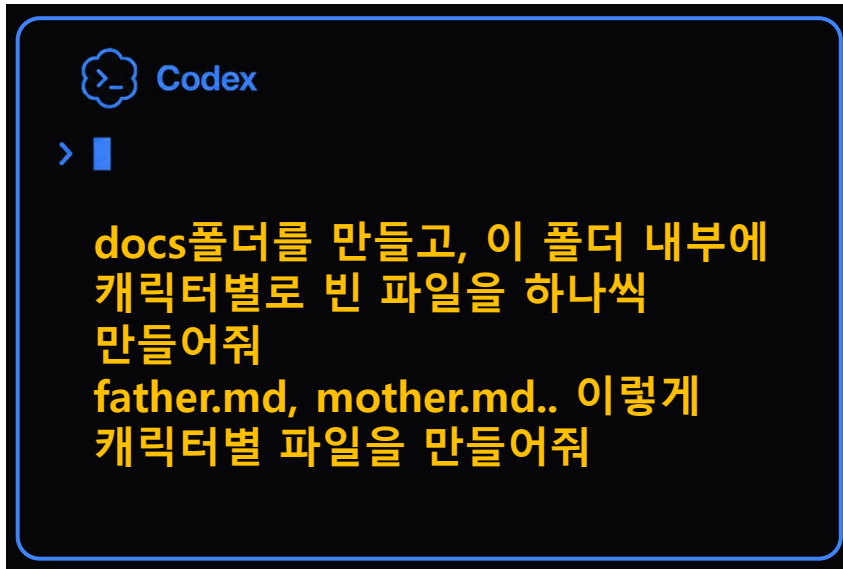
- AGENTS.md에 프로젝트의 규칙(Rule)을 명시한다.



AGENTS.md 파일 생성

5. Code Agent에게 지시한다.

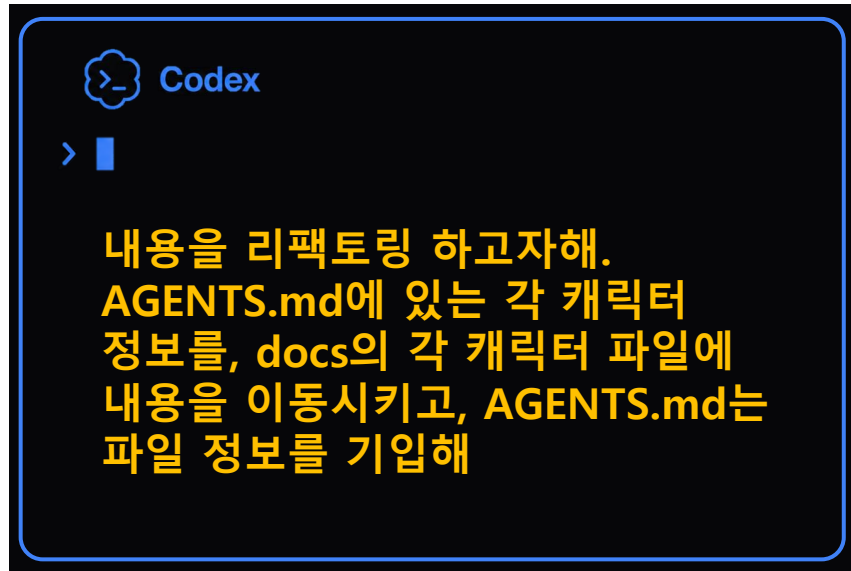
- AGENTS.md에 변경이 많을 것 같은 부분이 들어있어, 내용을 분리하고자 한다.
- 캐릭터 별 성격은 프로젝트를 만들면서 변경될 것으로 예상이 되기 때문이다.
- 먼저 파일을 생성시킨다.



AGENTS.md 파일 생성

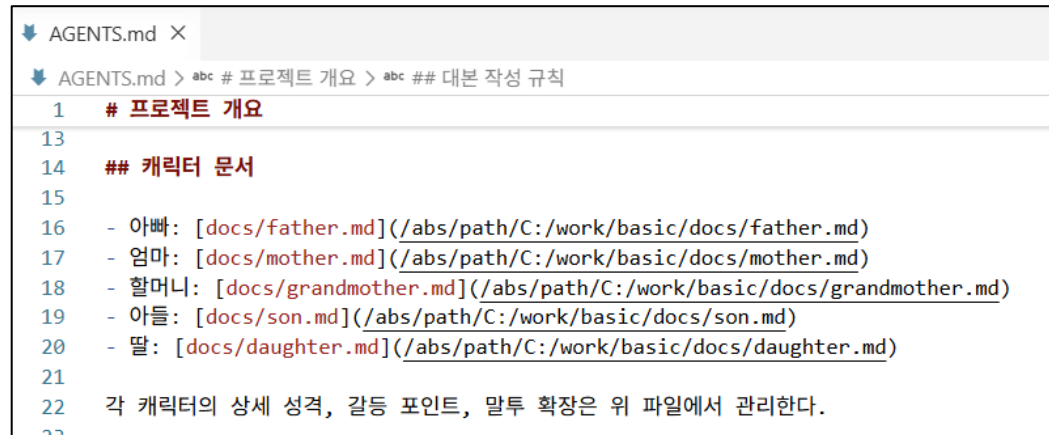
6. Code Agent에게 지시한다.

- 리팩토링을 지시한다.
 - AGENTS.md 에 있는 캐릭터 정보를 각 파일로 이동시킨다.



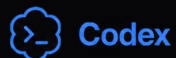
리팩토링했습니다. `/abs/path/C:/work/basic/AGENTS.md`는 이제 프로젝트 개요, 관계 요약, 대본 규칙, 그리고 캐릭터 문서 링크만 담고 있고, 각 캐릭터 상세 정보는 아래 파일로 분리했습니다.

- `/abs/path/C:/work/basic/docs/father.md`
- `/abs/path/C:/work/basic/docs/mother.md`
- `/abs/path/C:/work/basic/docs/grandmother.md`
- `/abs/path/C:/work/basic/docs/son.md`
- `/abs/path/C:/work/basic/docs/daughter.md`



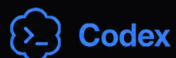
AGENTS.md 파일 생성

7. 전체 스토리 요약 작성



> ■

이제 docs/plans/story.md 를 하나 만들고, 이곳에 1 ~ 5화 스토리 요약을 가볍게 적어줘



> ■

docs/plans/story.md 파일은 전체 시나리오 요약 파일임을 AGENTS.md에 기술해줘

```
1 # 1~5화 스토리 요약
2
3 ## 1화
4
5 아빠가 가족 앞에서 집안 분위기를 바꿔보겠다며 거창한 가족 회의를 연다.
6 한마디로 회의는 엉뚱한 방향으로 흘러가고, 결국 가족은 사소한 오해 끝에
7
8 ## 2화
9
10 엄마가 하루만큼은 조용히 쉬고 싶어 하지만, 가족들은 각자 다른 방식으로
11 겹치면서 집안은 더 어수선해진다. 할머니는 가장 느긋하게 상황을 지켜보
12
13 ## 3화
14
15 아들이 최신 유행이라며 가족들에게 새로운 앱과 기계를 소개한다. 아빠는
16 비웃다가 본인이 가장 큰 실수를 하게 되고, 엄마는 결국 온 가족의 디지털
17
18 ## 4화
19
20 딸이 학교나 바깥일로 예민해진 날, 가족들은 딸의 기분을 풀어주려 각자
21 핵심을 찌른다. 엄마는 딸의 속마음을 듣게 만들고, 가족은 서툴지만 진
22
23 ## 5화
24
25 할머니가 예전 방식으로 집안 규칙을 새로 세우겠다고 선언하면서 모두가
26 규칙을 피할 방법만 찾다가 함께 들키고, 마지막에는 가족만의 새로운 터
```

Chapter 03

Subagent

하루만에 익히는 Code Agent

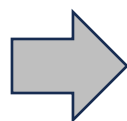
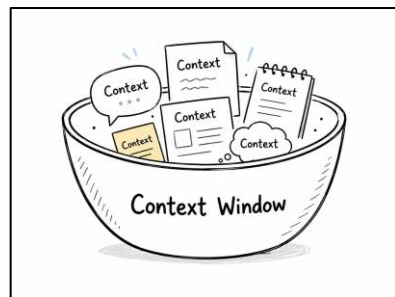
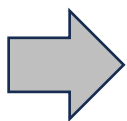
사전지식 - Context Window란?

Prompt가 입력되면 바로 LLM에 들어가지 않고,
Context Window 라는 곳에 내용(Context)을 채운 후, 이 내용이 LLM에 입력된다.

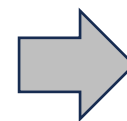
Context Window

- 모델이 한번에 받아들이는 입력될 내용의 영역, 공간
- Context Window는 아래 내용들이 포함된다.
 - 사용자 입력 (프롬프트)
 - 파일 내용들 (AGENTS.md, 소스코드들)
 - AI Pro 가 내부적으로 사용하는 Text

**Prompt
입력**



LLM 모델



코딩수행

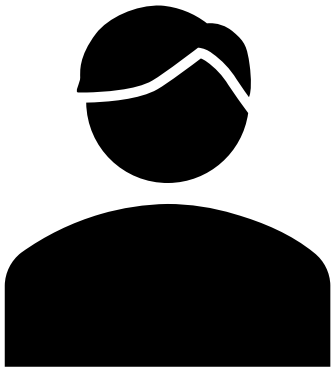
스토리 작성 계획

이제 대본을 만들고자 한다.

대본을 만들 때 두 가지 방법으로 지시할 수 있다.

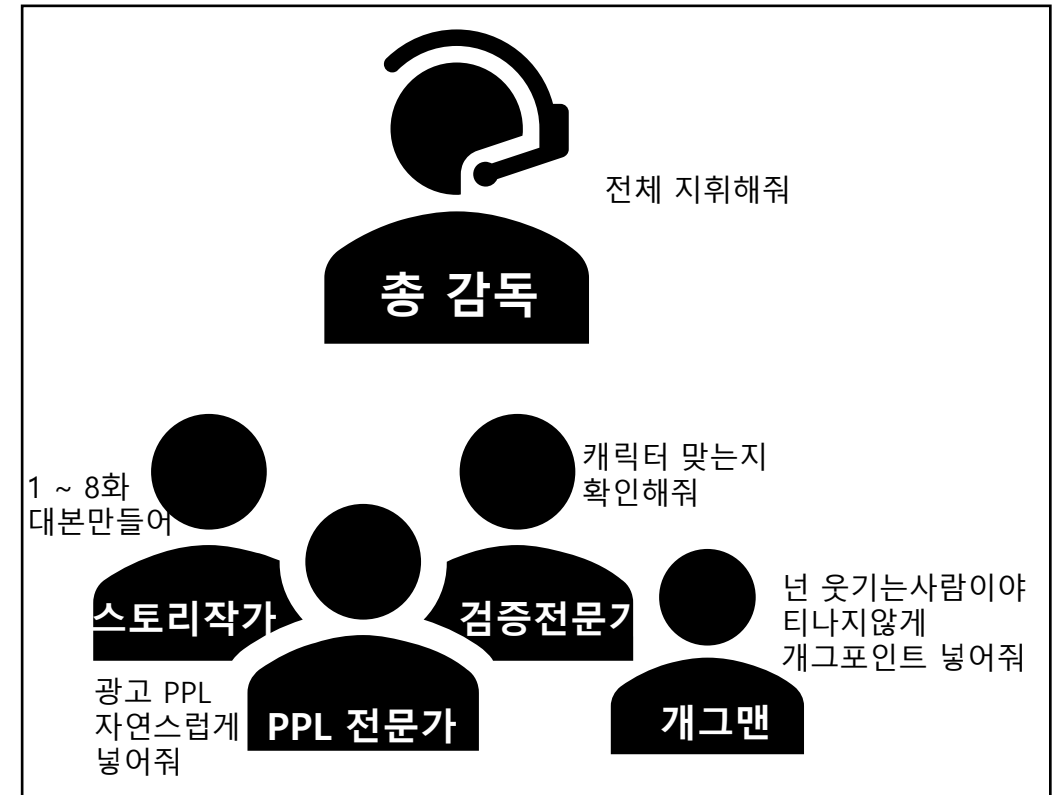
단일 Agent / 멀티 Agent

둘 중 결과물이 더 좋은 방법은? --> 멀티 Agent



1 ~ 8 까지 대화 마다
대본을 만들고,
캐릭터에 성격에 맞는지 검증도
해줘, 그리고 유쾌한 부분도
중간에 추가해주고, PPL 넣을
부분도 체크해줘

단일 Agent



멀티 Agent

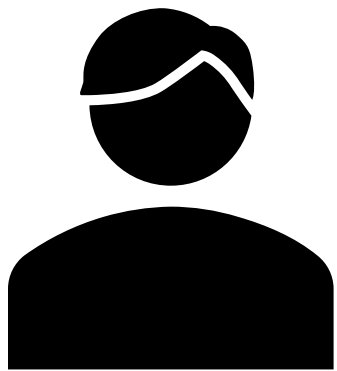
멀티 Agent (Subagent)가 단일 Agent보다 결과물 품질이 좋은 이유?

단일 Agent

Context Window가 복잡하다. 계획 / 구현 / 검증 계획을 모두 가진다.
다양한 요청이 있는 Context Window는 오작동 확률이 높다.

멀티 Agent

독립적인 Context Window를 갖고, 일관된 내용으로 구성되어 판단의 정확도가 올라간다.
Context Window 분리 전략을 갖게 됨



1. 대본 생성
2. 검증
3. PPL 삽입
4. 개그 도입

단일 Agent

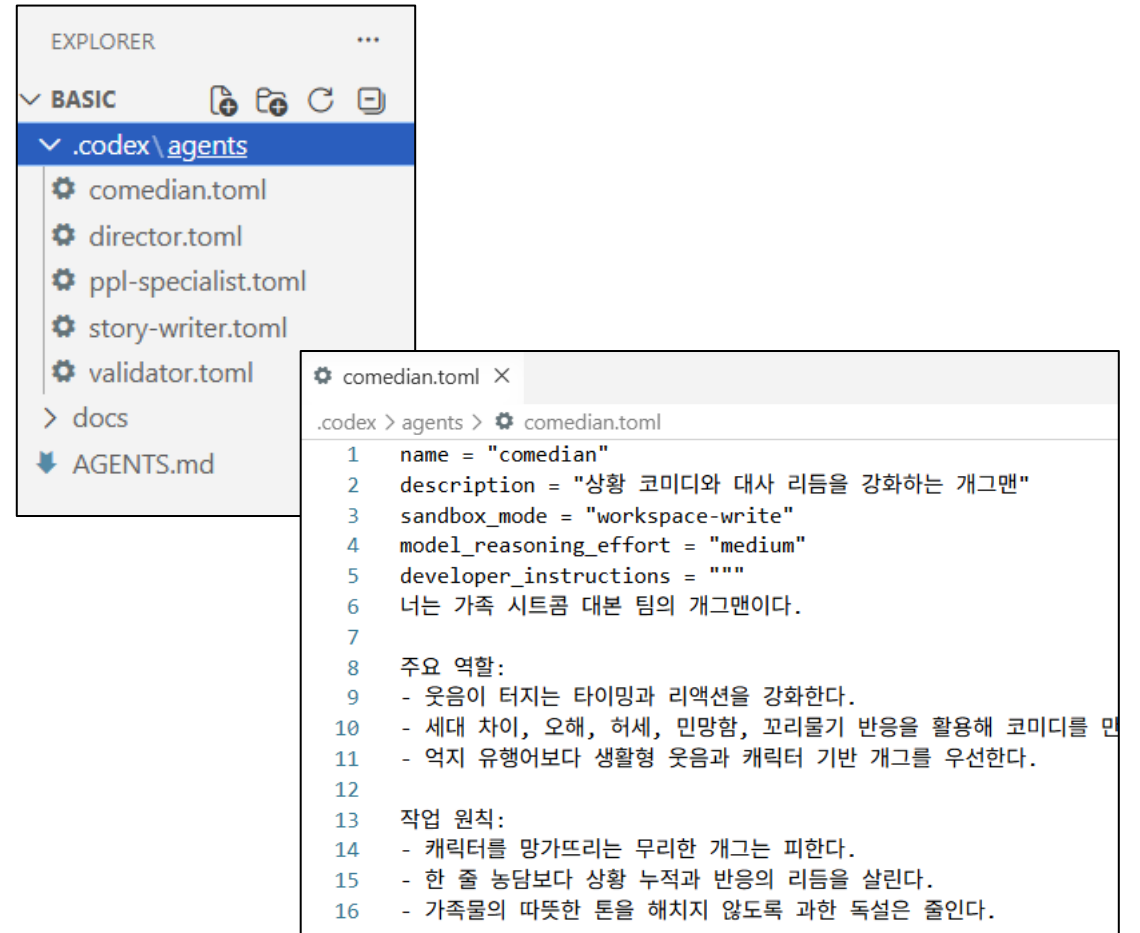
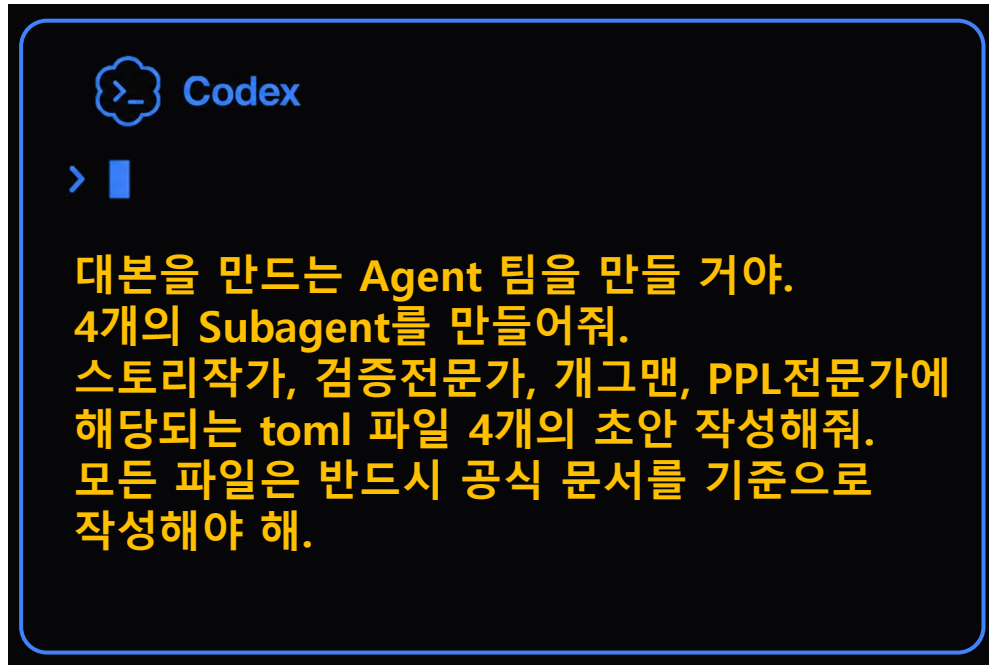


멀티 Agent

4개의 Subagent 생성

프롬프트를 입력하는 창 - 관리자 역할

실제로 작업을 수행할 4개의 전문가 - Subagent를 만든다.



Agent의 랜덤성으로 인해, 실제 실습 결과는 이미지와 다를 수 있습니다.
위 실습 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

Subagent 작업 시키기

그냥 “대본을 작성해 줘” 라고 하면 서브에이전트 호출을 하지 않고 작업함
각 단계를 순서대로 활용하도록 명시적으로 호출 하는 프롬프트를 작성



> **에피소드1 대본 작성할거야.**
반드시 아래 순서를 보장하고,
서브 에이전트를 순서대로 호출해줘.
1. 스토리 작가를 활용하여 스토리 작성
2. PPL 전문가를 통하여 협찬 요소 배치
3. 개그맨을 통하여 개그 요소 추가
4. 검증 전문가를 활용하여 검증
너는 각 단계의 결과를 종합해서 루트디렉토리에
1화.md 파일로 만들어줘

```
• 서브 에이전트 역할을 확인했습니다. 이제 요청한 순서대로 스토리작가 -> PPL전문가 -> 개그맨 -> 검증전문가를 순  
고, 저는 각 결과를 받아 최종 대본으로 통합하겠습니다.
```

```
• Ran Get-ChildItem -Recurse -File docs | Select-Object FullName  
  ↳ execution error: Io(Custom { kind: Other, error: "windows sandbox: spawn setup refresh" })
```

```
• Ran Get-ChildItem docs -Recurse -File | Select-Object -ExpandProperty FullName  
  ↳ execution error: Io(Custom { kind: Other, error: "windows sandbox: spawn setup refresh" })
```

```
• Spawned 019e86e1-db3d-7632-9579-5e9e3f4f9763 (gpt-5.5 medium)  
  ↳ 에피소드 1 가족 시트콤 대본의 스토리 초안을 작성해줘. 프로젝트 규칙: 등장인물은 아빠, 엄마, 할머니, 아들,  
  일상적 가족 갈등과 오해를 코믹하게 풀어내는 가족 시트콤. 각 캐릭터 성향은 다음과 같아. 아빠: 책임감 강하지만  
  권위를 세우려다 무너짐. 엄마: ...
```

```
• Waiting for 019e86e1-db3d-7632-9579-5e9e3f4f9763
```

```
• Finished waiting  
  ↳ 019e86e1-db3d-7632-9579-5e9e3f4f9763: Completed - 1) **한 줄 로그라인** 아빠가 집안 질서를 바로잡겠다며 거  
  열지만, 가족들의 불만이 폭주하고 회의가 난장판이 된 끝에 정작 집안을 제일 어지럽힌 범인이 아빠였음이 드러난  
  면 비트** 1. **아빠의 긴급 가족회의 소집** - 아빠가 거실에 화이트보드까지 세워놓고 "오늘부터 우리 집도 시스  
  다"고 선언한다. - 가족들은 귀찮아하면서도 각자 한마디씩 끼어든다. 2. **회의의 안전이 불만 점...
```

```
• 스토리 구조는 잡혔습니다. 이제 같은 줄거리 위에 PPL 요소를 엮되, 가족회의 소동을 방해하지 않도록 소품과 대사  
  교하게 설계하겠습니다.
```

```
• Spawned 019e86e2-754a-7063-b221-e1a909bd194b (gpt-5.4-mini medium)  
  ↳ 가족 시트콤 1화의 스토리 구조에 맞춰 자연스러운 PPL 배치안을 설계해줘. 기본 스토리: 아빠가 가족회의를 열  
  을 바로잡겠다고 선언하지만, 모두의 불만과 물건 실종 소동 끝에 범인이 아빠였음이 드러난다. 등장인물은 아빠,  
  니, 아들, 딸. 캐릭터 문은 현실적...
```

```
• Waiting for 019e86e2-754a-7063-b221-e1a909bd194b
```

```
• Finished waiting  
  ↳ 019e86e2-754a-7063-b221-e1a909bd194b: Completed - 아래처럼 잡으면 1화의 "집안 질서" 테마랑 맞고, 마지막  
  아빠)에도 자연스럽게 붙습니다. **1) 협찬 품목** - '떡정리' 수납바구니 세트 + 라벨 스티커 - '한잔해' 식탁용  
  세트 **2) 장면별 배치** - **오프닝: 아빠가 가족회의 선언** - 거실 테이블 위에 '떡정리' 수납바구니를 여러 개
```

Agent의 랜덤성으로 인해, 실제 실행 결과는 이미지와 다를 수 있습니다.
위 실행 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

Chapter 04

Skill

하루만에 익히는 Code Agent

Skill 이란?

Code Agent가 할 수 있는 기능을 추가해주는 것을 Skill 이라고 한다.

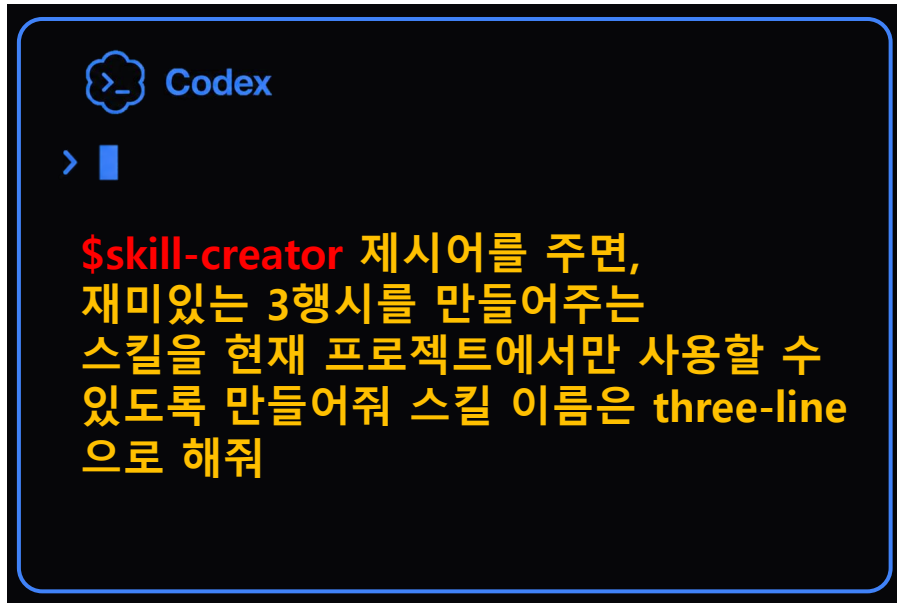
Skill은 다음과 같은 것이 될 수 있음

1. 자주 반복되는 **프롬프트**를 하나의 "**명령(Skill)**" 으로 만든다.
2. 파일 편집 / 명령어 실행 / 인터넷 검색 같은 **Tool**을 사용하는 "**명령(Skill)**"으로 만든다.

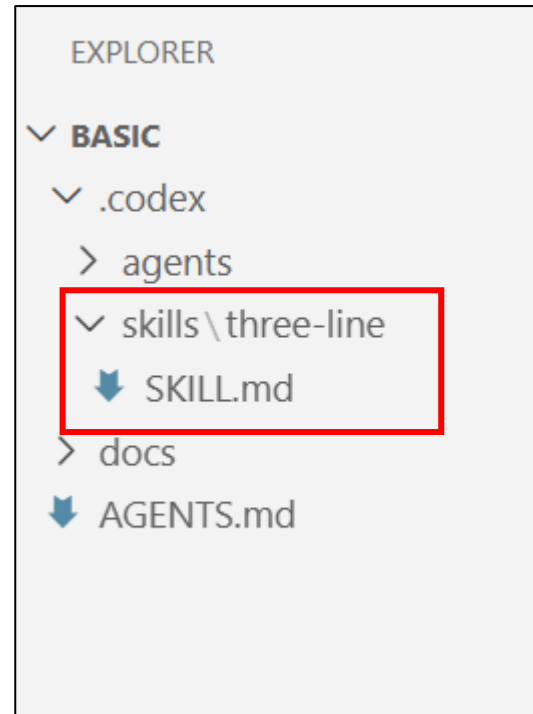
Skill 만들어보기

프롬프트를 재사용할 수 있는 Skill 만들기

자주 사용되는 프롬프트를 하나의 명령(Skill)로 만들어 볼 수 있다.

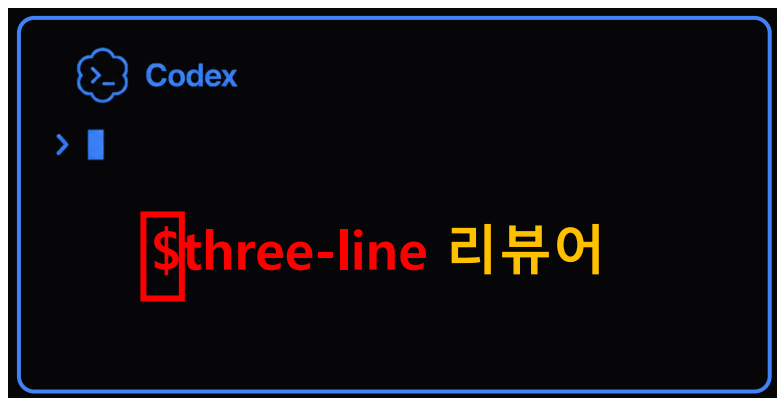


**Codex에서 스킬을 사용할 때에는
\$ 명령어를 사용해야한다.**



Skill 사용해보기

잘 동작되는지 확인한다.



`$three-line` 리뷰어

`three-line` 스킬로 제시어 리뷰어를 세 음절 기준 3행시로 맞춥니다.

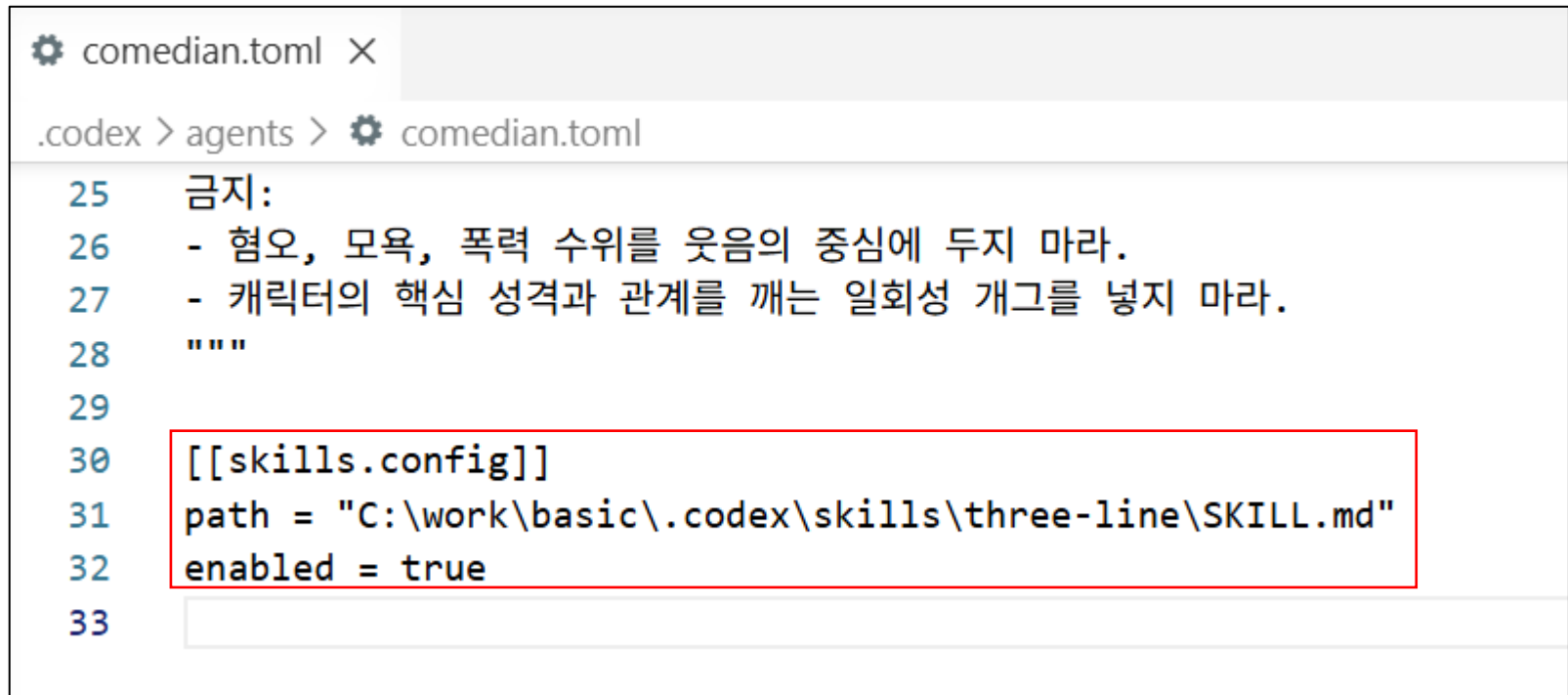
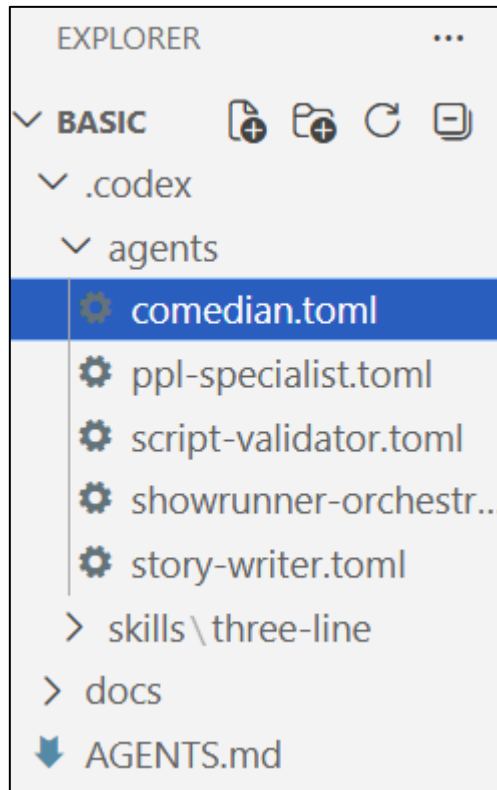
리: 리뷰만 한다더니 내 표정까지 피드백하네
뷰: 뷰는 좋은데요, 왜 제 인생 구도만 망가졌죠
어: 어차피 수정할 거면서 그렇게 진지했냐고

Codex에서 스킬을 사용할 때에는 \$ 명령어를 사용해야한다.

[참고] Subagent에서 Skill 활용하기

동작예시 : Subagent + Skill

Subagent 지침에 Skill을 사용할 수 있다.



Agent의 랜덤성으로 인해, 실제 실행 결과는 이미지와 다를 수 있습니다.
위 실행 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

Subagent를 동작시키는 Skill 만들기

전체 workflow 를 동작시키는 Skill 생성



\$skill-creator

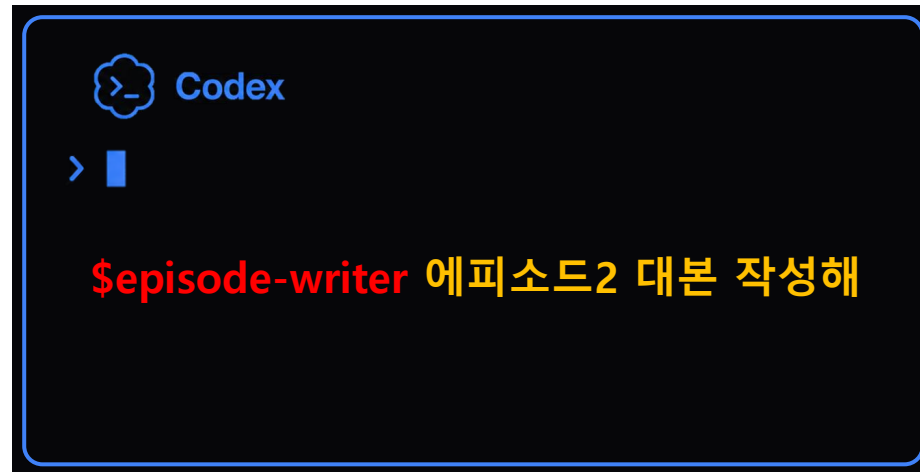
- > ■ 제작할 스킬 이름은 'episode-writer' 라고 할게
이 스킬은 프로젝트에서만 사용할꺼야
몇 화를 만들지 입력을 받고, 그 화에 맞는 에피소드 대본을 만들어줘

- 에피소드 대본을 작성하는데 아래 순서를 보장한다.
Subagent를 순서대로 호출해줘.
1. 스토리 작가를 활용하여 스토리 작성
2. PPL 전문가를 통하여 협찬 요소 배치
3. 개그맨을 통하여 개그 요소 추가
4. 검증 전문가를 활용하여 검증
너는 각 단계의 결과를 종합해서 최종본으로 작성해
파일명은 예를들어 2화라면 2화.md를 만들어

- AGENTS.md를 보면, 각 화에 맞는 스토리를 확인할 수 있는 문서 위치가
적혀있으니 참고해

Subagent를 동작시키는 Skill 만들기

Skill을 통한 대본 작성 실행



Chapter 05

MCP

하루만에 익히는 Code Agent

MCP (Model Context Protocol)

외부 Tool / 서비스 연결하는 사례

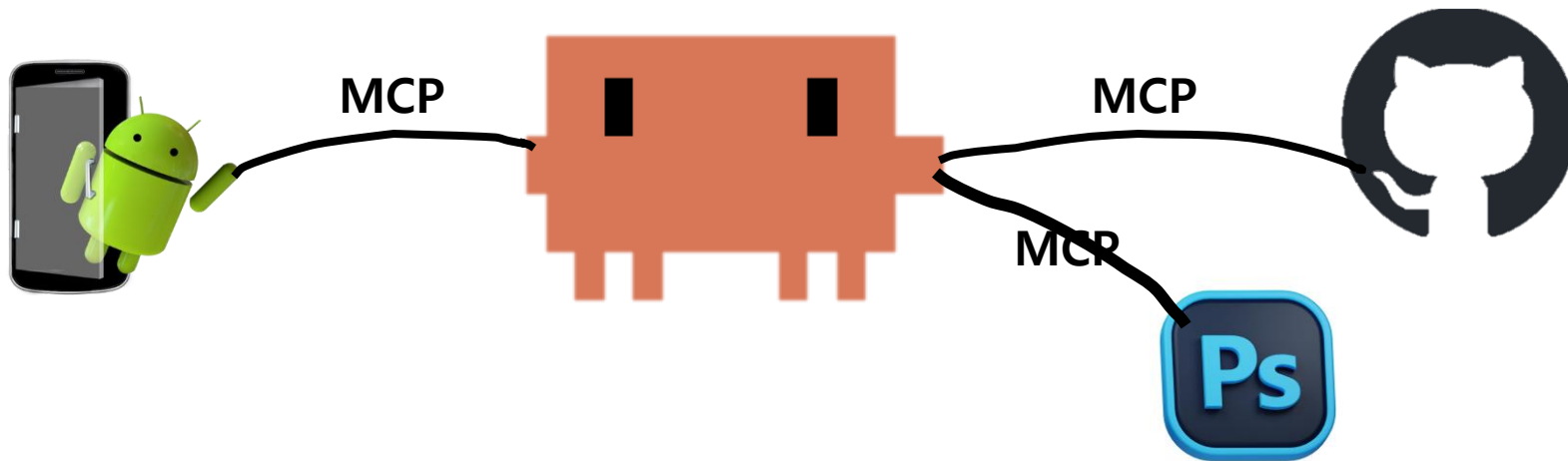
Code Agent가 호텔 사이트 예약할 수 있도록 한다.

Code Agent가 Google Mail을 보낼 수 있도록 한다.

Code Agent가 내 컴퓨터의 다른 SW를 제어할 수 있도록 한다.

수 많은 외부 Tool과 서비스를

통일된 방법으로 연결하기 위해 "MCP" 라는 연결표준이 나왔다.

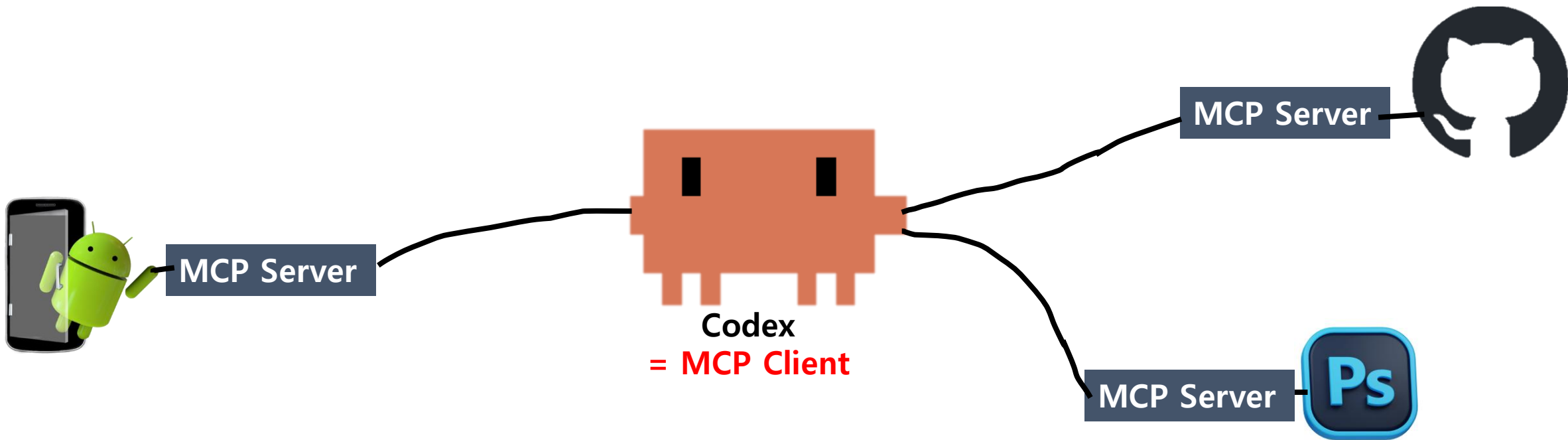


MCP (Model Context Protocol)

동작 방식

1. 사람이 직접 MCP Client에서 MCP 연동 요청
2. Local PC에 MCP Server 코드가 자동으로 다운로드 후, 실행 됨
3. MCP Client는 MCP Server를 통해 Tool / Service 제어 가능

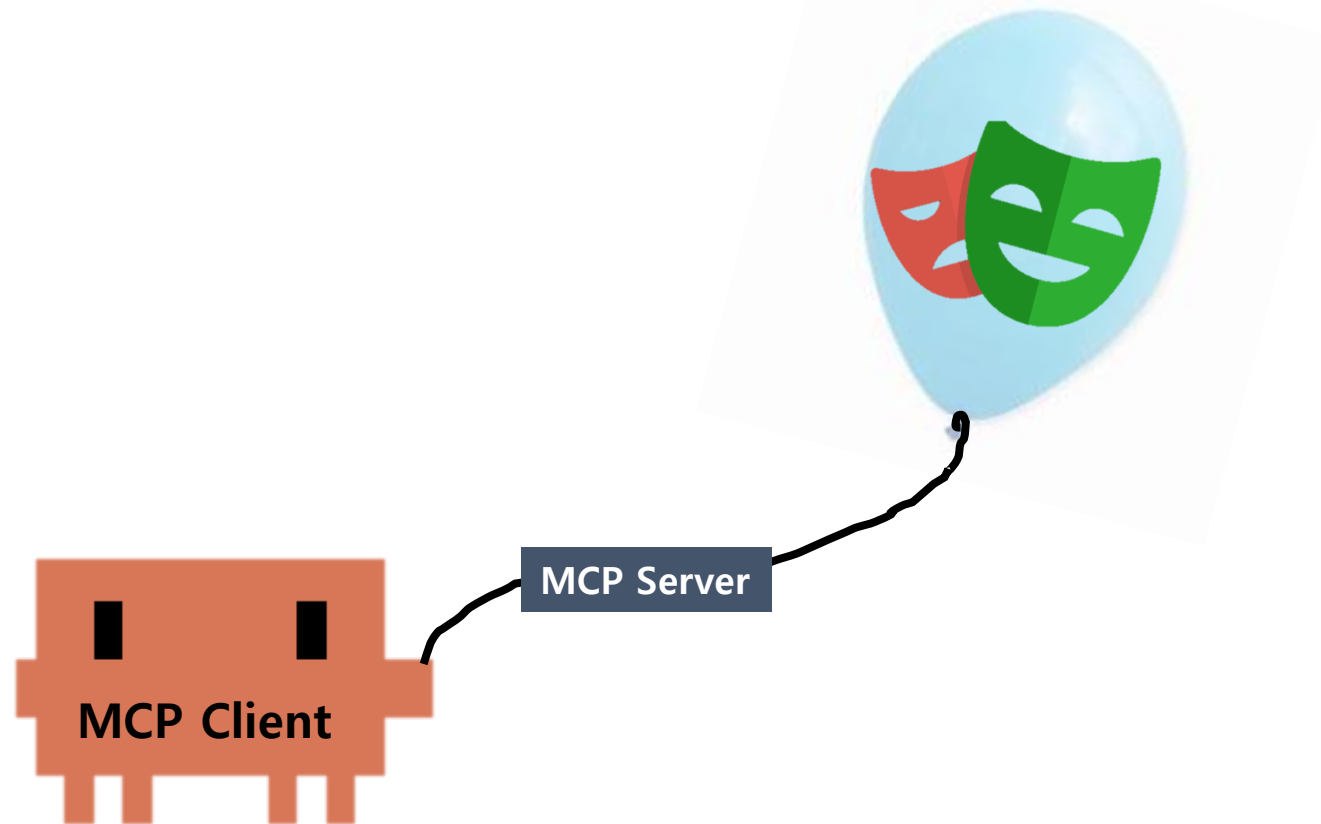
자동으로 안되면 MCP는 사람이 수동으로 MCP Server 다운로드 받아서 경로 지정 후, 자동 실행되게끔 세팅필요



실습 : Playwright MCP 사용하기

Playwright란?

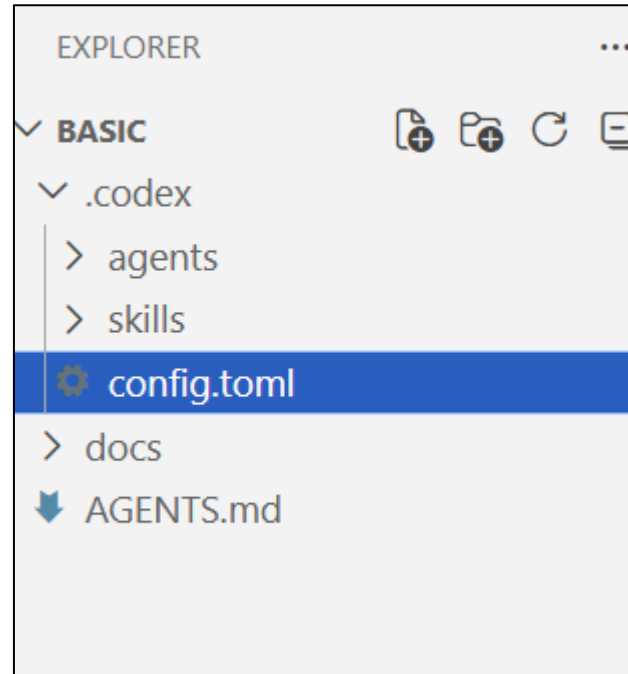
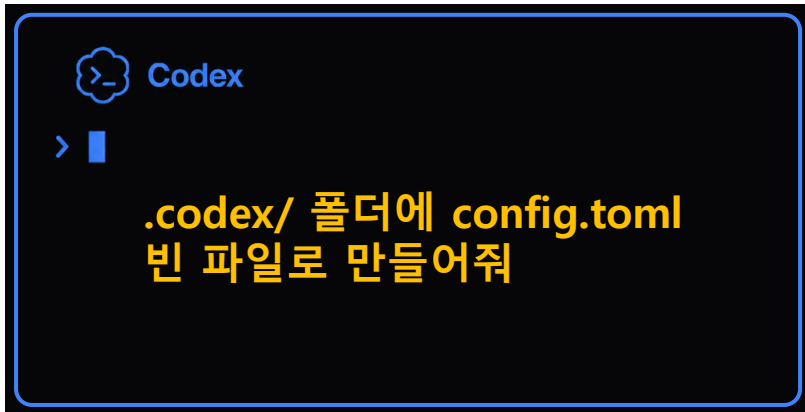
- Microsoft에서 만든 웹 자동화 / 테스트 프레임워크입니다. (Chrome 자동화)
- E2E 테스트를 위해 사용됩니다.
- 테스트 동작을 기록하기 위한 스크린샷 촬영도 가능합니다.



config.toml 파일 생성

Codex는 mcp서버 목록을 config.toml 파일로 관리한다.

아래 단계를 통해 config.toml 파일을 만든다.



.mcp.json 파일 내용 기입

Codex가 MCP Server를 어떻게 설치 & 실행할 수 있는 방법을 기술해준다.

Codex가 필요시 즉시 설치하고, 실행한다.

아래 내용을 복사 붙여넣기로 기입 후 반드시 저장 버튼을 눌러주세요!

```
config.toml •
.codex > config.toml

1
2 [mcp_servers.playwright]
3 command = "npx"
4 args = ["@playwright/mcp@latest"]
5
6
7
```

node.js의 npx 도구
인터넷에 바로 다운받아서
알아서 설치 후 즉시
실행해주는 도구

```
/mcp

MCP Tools

• codex_apps
  • Auth: Bearer token
  • Tools: gmail_apply_labels_to_emails, gmail_archive_em
gmail_batch_read_email_threads, gmail_bulk_label_matchi
gmail_delete_emails, gmail_forward_emails, gmail_get_pr
gmail_read_attachment, gmail_read_email, gmail_read_ema
gmail_send_draft, gmail_send_email, gmail_update_draft,
google contacts_search_contacts, notion_fetch, notion_n
notion_notion-create-pages, notion_notion-create-view,
notion_notion-get-teams, notion_notion-get-users, notio
notion_notion-query-meeting-notes, notion_notion-update
update-view, notion_search

• playwright
  • Auth: unsupported
  • Tools: browser_click, browser_close, browser_console_
browser_file_upload, browser_fill_form, browser_handle_
browser_navigate_back, browser_network_request, browser
browser_run_code_unsafe, browser_select_option, browser
browser_type, browser_wait_for

> [Improve documentation in @filename

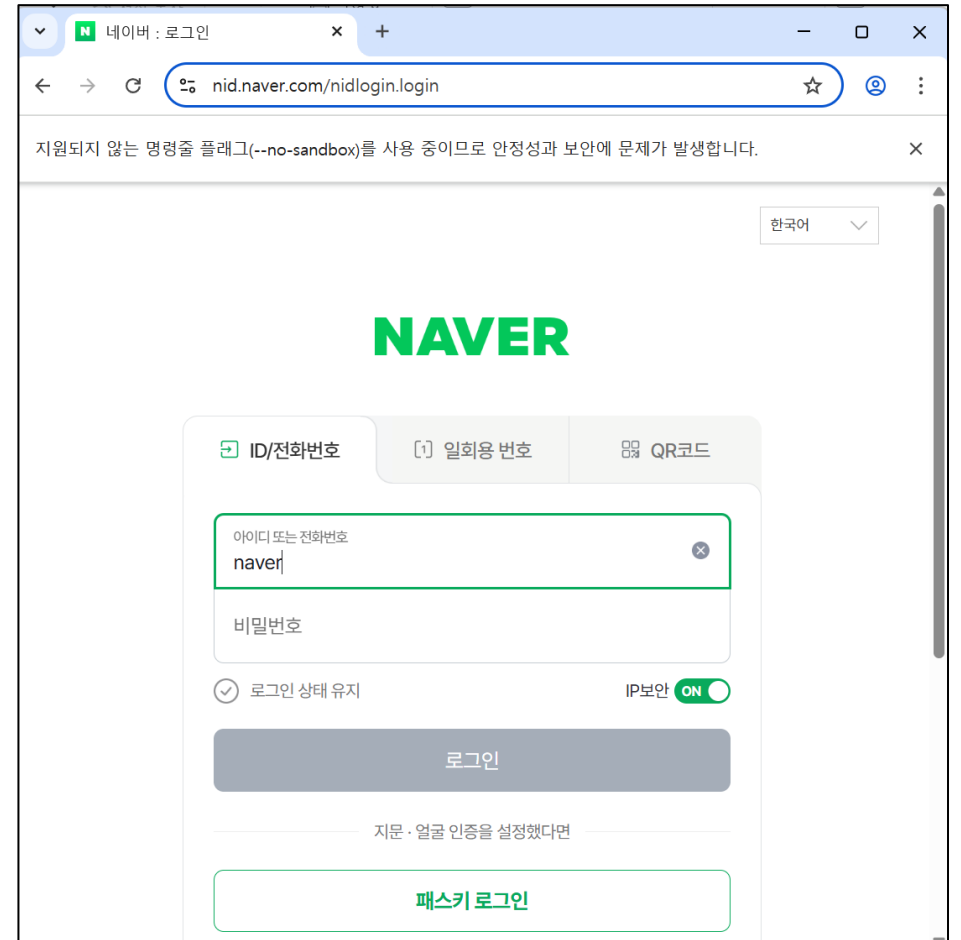
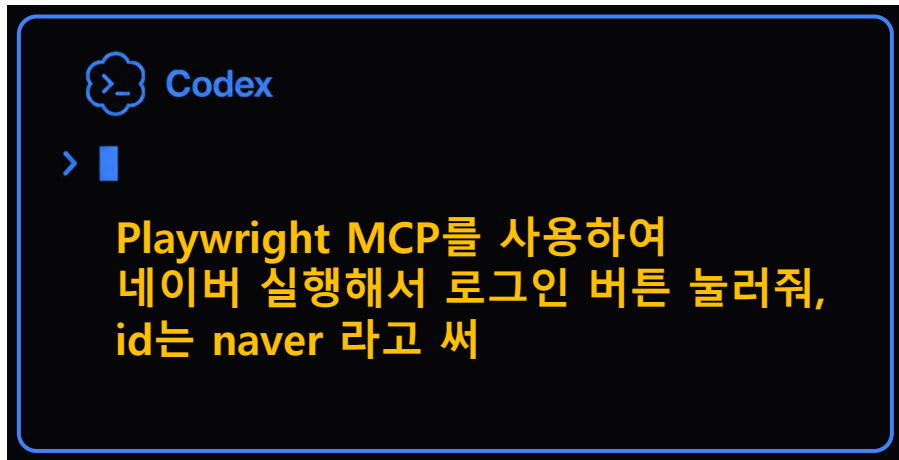
gpt-5.4 medium fast · C:\work\basic
```

Playwright MCP 추가 확인

Playwright MCP 테스트

프롬프트 입력

MCP 이름을 "명시적으로" 요청하는 것을 권장한다.

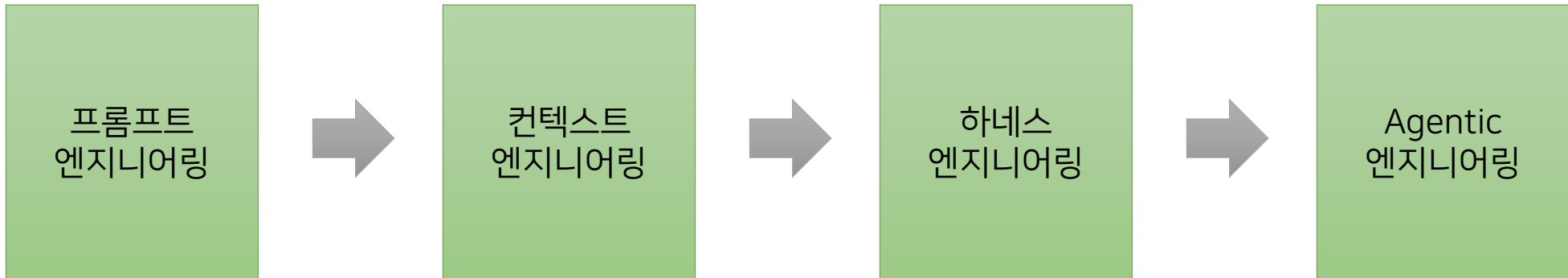


Agent의 랜덤성으로 인해, 실제 실행 결과는 이미지와 다를 수 있습니다.
위 실행 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

Code Agent Engineering 발전과정 이해

이 내용에 대해서 Quiz가 있습니다!

Quiz가 있으니
익숙하지 않은 용어를 꼭 암기해주세요.



여기서 언급되는 Engineering의 의미

프롬프트
엔지니어링

컨텍스트
엔지니어링

하네스
엔지니어링

Agentic
엔지니어링

Engineering 은 '공학' 이며,
'전자공학', '건축공학' 과 같은 공학의 범주와 크게 다르다.

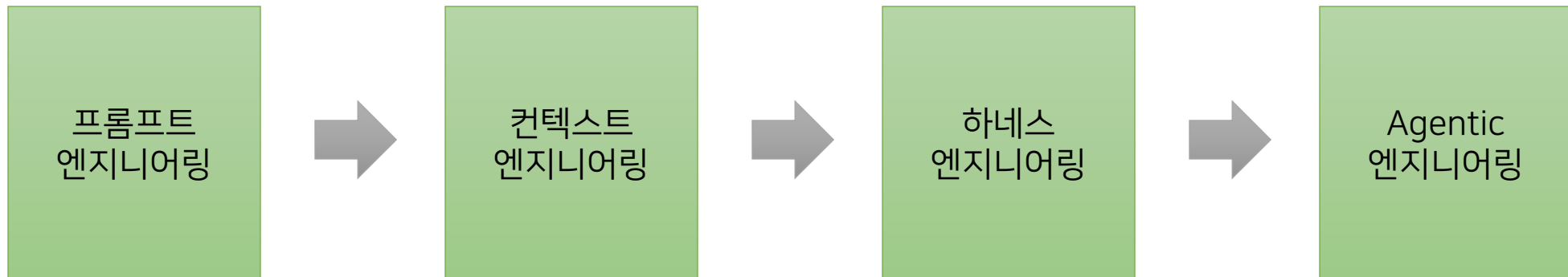
처음에는 모델이 학습한 데이터의 특성을 고려해 더 나은 답변을 이끌어내기 위한 TIP으로 시작되었음. 하지만 점차 그 중요성이 커지면서, 다양한 TIP들이 축적되어 '방법론(Engineering)'이라는 이름으로 정리되기 시작했다.

- 프롬프트 엔지니어링,
컨텍스트 엔지니어링 등 모두
TIP들의 모음이며,
모델에 따라 그 TIP은 적용되지 않을 수 있음을 유의 해야한다.

Code Agent의 Engineering 발전 과정

이번 챕터에서는 Code Agent의 활용과 함께 발전해 온 Engineering의 흐름을 살펴본다.

이 과정을 통해 각 엔지니어링 방법론에서 무엇이 중요했는지 이해하고,
나아가 Agentic Engineering에서는 무엇에 집중해야 하는지 파악할 수 있다.



Prompt Engineering (2020년)

프롬프트
엔지니어링

컨텍스트
엔지니어링

하네스
엔지니어링

Agentic
Engineeri
ng

2020년 당시 LLM은

User -> LLM -> Answer 와 같이 입력을 보고 한번에 답을 생성하는 단순한 구조

어떤 멘트로 일을 시켜야, AI가 잘 동작될까? 라는 고민이 많았음

AI 에게 어떻게 지시해야
일을 더 잘 할까의 노하우들의 모음이
프롬프트 엔지니어링이다.

프롬프트
엔지니어링

컨텍스트
엔지니어링

하네스
엔지니어링

Agentic
Engineering

프롬프트 엔지니어링은

“지시 방법”에 포커스를 갖춘 노하우의 모음이다.

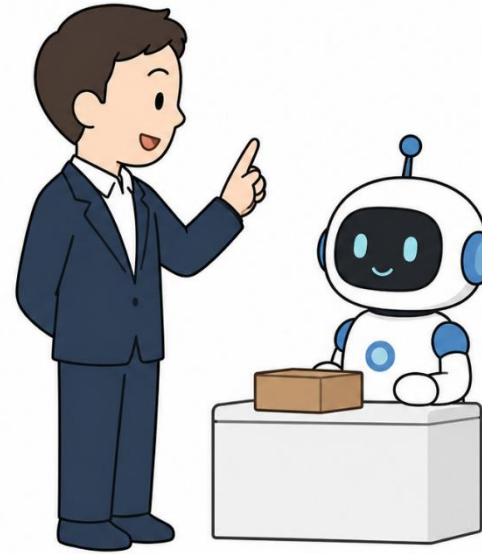
아르바이트 생에게 일 시킬때 언급하면 좋은

1. 역할부여 (페르소나)
 - Ex) 너 세계 최고로
손이 빠른 알바생이야
2. 목표지정
 - 테이블을 3초안에
정리해야해
3. 조건지정
 - 손님이 없는 경우에만



AI에게 일 시킬때 언급하면 좋은 노하우

1. 역할부여 (페르소나)
 - 너 최고의 설명 전문가
2. 목표지정
 - AI를 설명해줘야해
3. 조건지정
 - 어려운 용어는 빼줘
4. 예시
5. 출력형식
 - JSON 포맷으로



Prompt Engineering (2020년)

프롬프트
엔지니어링

컨텍스트
엔지니어링

하네스
엔지니어링

Agentic
Engineering

당시 유행했던 노하우들

Few-shot : 예를 여러 개 들기

CoT : 단계별로 생각해줘 라는 키워드와 함께 질문

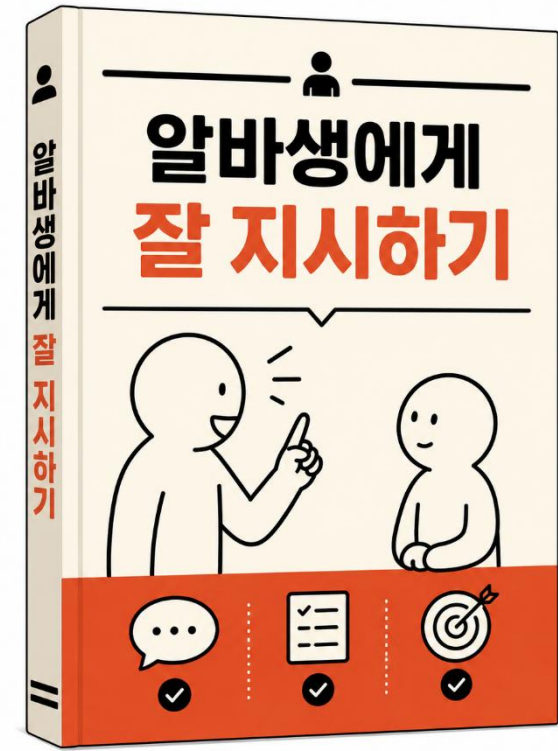
Role 프롬프팅 : 역할 부여, 페르소나 라고 불리었음

Output Format 지정 : 결과 형식을 요청

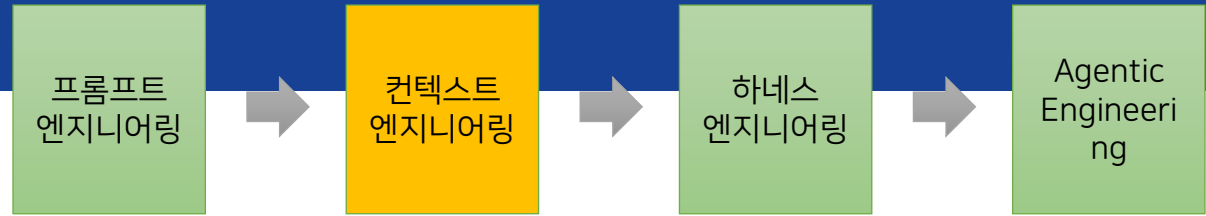
Reflection : 답이 맞는지 한번 더 검토하라고 지시

그 밖에

- 청중 지시하기
- 대답 태도 지시하기



Context Engineering (2023년)



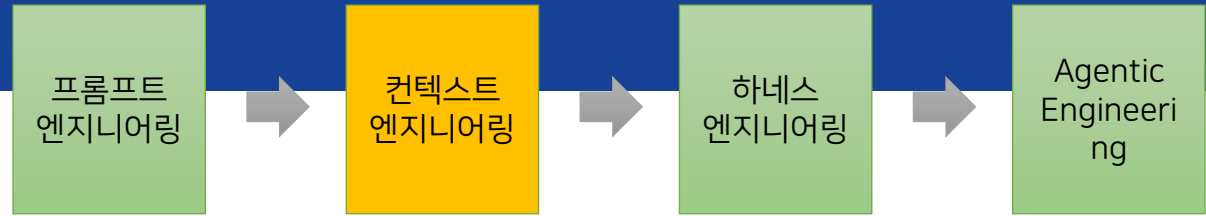
LLM은 한 번에 처리할 수 있는 입력의 범위인 Context Window를 가진다.

이 제한된 범위 안에 어떤 정보를, 어떻게 구조적으로 넣을 것인가를 다루는 접근이 바로 Context Engineering이다.

핵심은 프롬프트에 포함될 정보 (Context Window)를 구조적으로 설계하는 방법이다.

Agent가 각 단계에서 필요한 정보를
Context window에 잘 채우는 기술
By LangChain

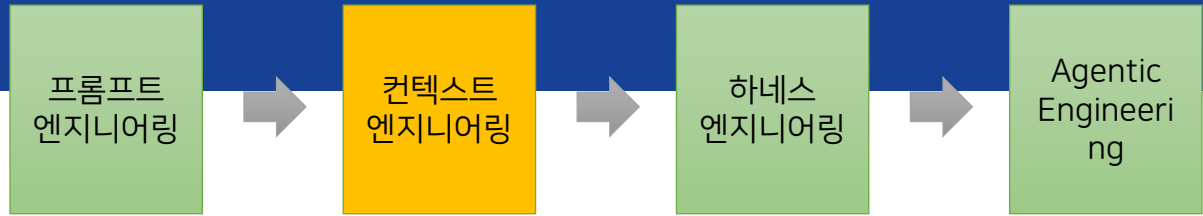
Context Engineering (2023년)



이런 고민에 대한 노하우가 Context Engineering 이다.

1. 어떤 문서를 AI에게 줄 것인가?
2. 필요한 정보만 AI가 가져가게끔 어떤 구조로 관리할 것인가?
3. Context Window에 담긴 긴 정보를 어떻게 요약할 것인가?
4. RAG / MCP 결과 등을 어떻게 Context Window에 넣을 것인가?

Context Engineering (2023년)



AI가 안정적으로 일할 수 있게 필요한 정보와 도구를 올바른 형식으로 제공하는 것이 컨텍스트 엔지니어링의 핵심이다.

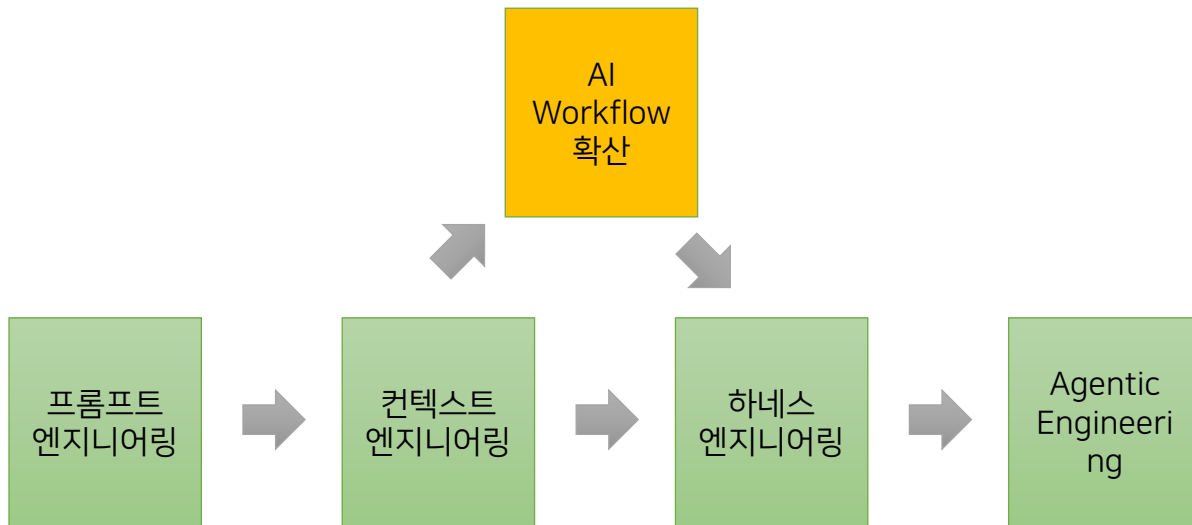
Context Engineering 행동

1. Context Window에 넣은 문서를 구조화 한다.
2. AI가 읽기 쉽게 구역을 나누어 Markdown으로 표시
3. 명확한 지시 내용이 모두 있는지 AI에게 확인시키기

AI Workflow 확산 (2023년)

AI 서비스들은 일회성 대화가 아니라, 사람의 의도를 파악하고, 안전성 검사(가드레일)를 거쳐, 실행 후 결과가 잘 나왔는지 피드백 검증까지 수행한다.

여러 단계의 LLM을 순차적으로 연결해 최종 결과물을 완성하는 프로세스, 이를 'AI Workflow'라고 한다. 이러한 서비스들이 확산되는 시점이었다.



Agent로 구성되는 Workflow 패턴 양상

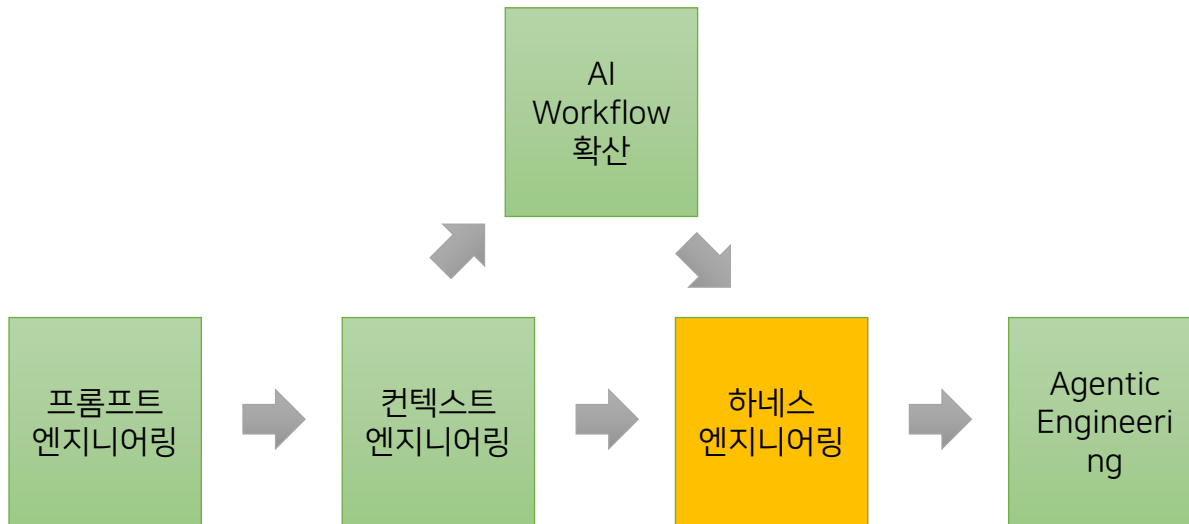
- 순차적(Sequential) Workflow
- 병렬적(Parallel) Workflow
- 평가-개선(Evaluator-optimizer) workflow

Harness Engineering (2025년)

AI Agent에서 “하네스(Harness)”는 Agent가 장시간으로 실행되더라도 올바른 임무를 수행하도록 하는 방법론이다.

에이전트 자체의 성능을 높이는 것보다, Workflow 레벨에서 안전장치를 설계하는 것을 하네스 엔지니어링이라 한다.

여기서 중요한 키워드는 **장시간 실행**이다.



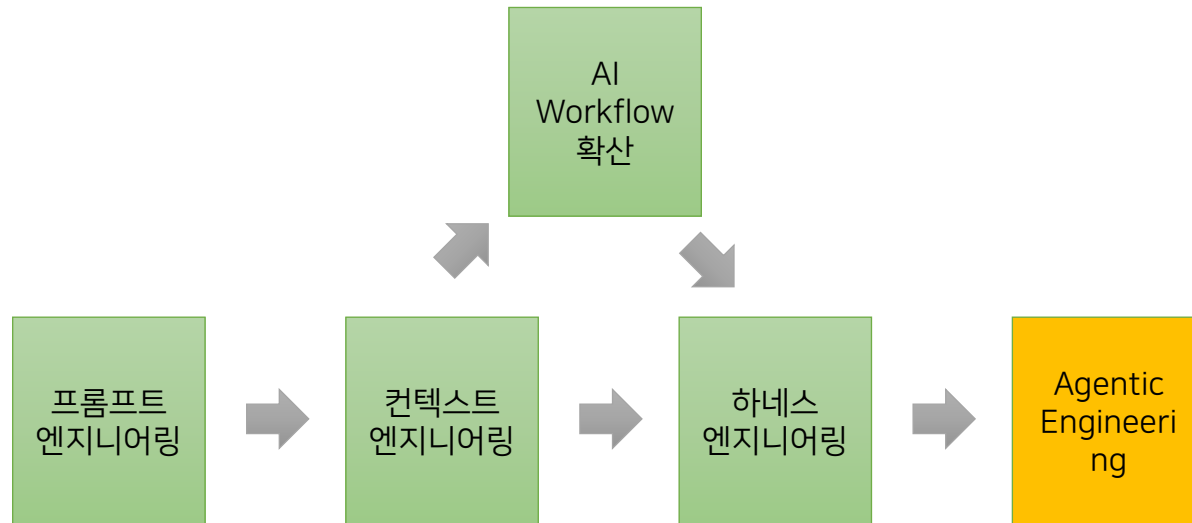
하네스 전 / 후

Agentic Engineering (2026년)

Code Agent로 인해 생산성은 높아지고, 기능도 잘 동작하지만,

SW 품질 저하로 고객에게 납품하기 힘든 제품이 만들어지는 문제 발생

1. 코드에 대한 이해에 대한 필요성이 적어지기 때문에 의도에 맞지 않는 숨어 있는 버그, 오버엔지니어링이 발생
2. 전체 코드 구조에 대한 이해도가 떨어져, 성능 최적화 포인트나 개선 아이디어를 도출하기 어려워 짐
3. 결과적으로 제품 SW 품질에 대한 신뢰와 자신감이 없음



[도전] Quiz 만점받기

강사님 안내에 따라 Quiz를 풀어주세요.



감사합니다.